AI-Based Electricity Demand Forecasting with Scenario Analysis

Component 3 — Demand Forecasting & Scenario Analysis Module Final Year Research Project | Sri Lanka Institute of Information Technology (SLIIT)

Research Information

Student Name	Dewmini D.M.D.P
Student ID	IT22630216
Institution	Sri Lanka Institute of Information Technology (SLIIT)
Supervisor	Dr. Nathali Silva
Research Component	Component 3 — Electricity Demand Forecasting with Scenario Analysis
Academic Year	2025 / 2026

Abstract

This research component develops an AI-based electricity demand forecasting system that predicts future electricity consumption and performs "what-if" scenario analysis to support decision-making for energy planners and grid operators. The system trains a Random Forest Regressor on historical electricity demand data incorporating weather, economic, and temporal features to achieve near-perfect forecasting accuracy (Test $R^2 = 0.9998$).

The trained model is deployed through a RESTful FastAPI backend and an interactive React dashboard, enabling users to submit baseline conditions, apply scenario modifications (such as heat waves, cold spells, price shocks, and economic booms), and receive demand predictions, risk assessments, and actionable recommendations in real time.

Research Problem & Motivation

Electricity demand is highly sensitive to external factors including temperature, economic activity, and public events. Grid operators and utility planners require accurate demand forecasts and the ability to model the impact of unexpected conditions before they occur. Traditional forecasting tools lack scenario planning capabilities that would allow planners to assess risk and prepare responses proactively.

This component addresses that gap by:

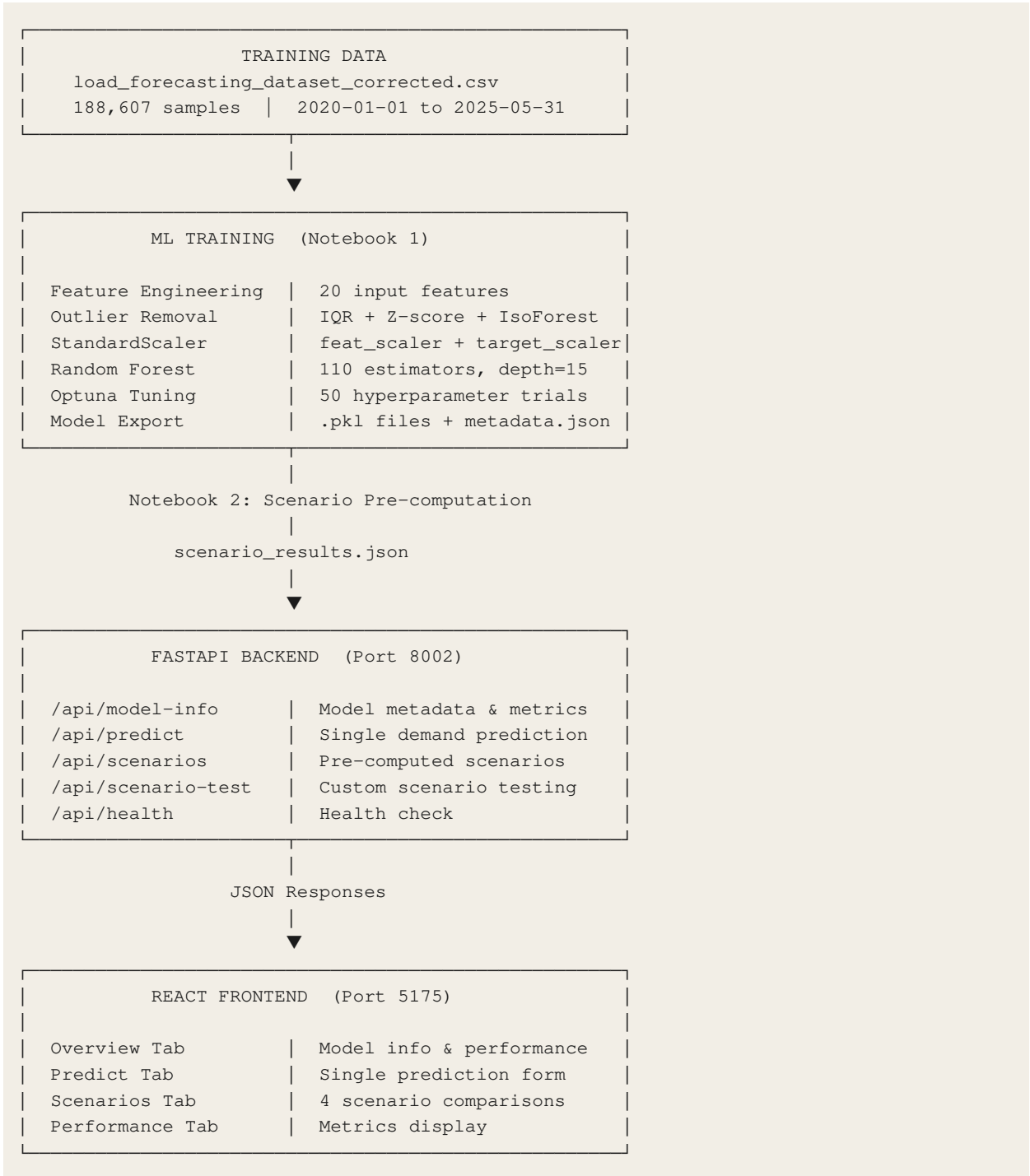
- Training a high-accuracy ML model on five years of electricity demand data with rich feature engineering
- Exposing scenario analysis through a simple API — allowing any condition to be modified and its demand impact assessed immediately
- Classifying demand change magnitude into risk levels (Low, Medium, High) with automated recommendations
- Presenting results through an interactive dashboard accessible to non-technical stakeholders

Research Objectives

1. Engineer a comprehensive 20-feature dataset combining weather, economic, temporal, and interaction variables from raw electricity consumption data
2. Train an optimised Random Forest Regressor achieving $R^2 > 0.99$ on unseen test data
3. Design and implement a scenario analysis engine supporting arbitrary parameter modifications
4. Classify demand change magnitude into risk levels with automated recommendations
5. Deploy the forecasting system as a REST API with full Swagger documentation
6. Build an interactive React dashboard for real-time prediction and scenario exploration

---

System Architecture



Technology Stack

Backend

Technology	Version	Role
Python	3.9+	Core language
FastAPI	0.104.1+	REST API framework
Uvicorn	0.24.0+	ASGI server
scikit-learn	1.8.0+	Random Forest, StandardScaler
pandas	2.1.0+	Data processing
numpy	1.26.0+	Numerical computation
scipy	1.11.0+	Statistical functions
joblib	1.3.2+	Model serialisation / loading

Frontend

Technology	Version	Role
React	18.2.0	UI component framework
Vite	5.4.0	Build tool and dev server
Tailwind CSS	3.4.3	Utility-first styling
Axios	—	HTTP client for API communication

ML Model Details

Algorithm

Random Forest Regressor - Estimators: 110 trees - Max depth: 15 - Hyperparameter optimisation: Optuna (50 trials) - Trained samples: 150,885 - Test samples: 37,722

Model Performance

Metric	Training Set	Test Set
R^2 Score	0.9999	**0.9998**
MAE	1.3576 kW	**1.5621 kW**
RMSE	1.9819 kW	**2.5084 kW**

Input Features (20 Total)

Category	Features
Weather	Temperature, Humidity, Wind Speed, Rainfall, Solar Irradiance
Economic	GDP, Per Capita Energy Use, Electricity Price
Temporal	Hour of Day, Month, Day of Week, Public Event
Derived / Interaction	TemperatureSquared, IsHotDay, IsColdDay, TempHumidityInteraction, WindRainInteraction, IsWeekend, IsPeakHour, IsBusinessHour

Top Feature Importances

Feature	Importance
Temperature	38.9%
TemperatureSquared	38.3%
IsHotDay	11.0%
TempHumidityInteraction	10.3%

Scenario Analysis

Four pre-defined demand scenarios are evaluated against a baseline:

Scenario	Description	Parameters Modified
Heat Wave	Extreme summer heat conditions	Temperature +8°C, Humidity –15%
Cold Spell	Severe winter conditions	Temperature –10°C, Humidity +20%
Price Shock	Electricity price doubled	Electricity Price ×2
Economic Boom	Rapid economic growth	GDP +20%, Per Capita Energy Use +10%

Risk Classification

Risk Level	Demand Change	Recommended Action
Low	< 5%	No action needed
Medium	5% – 10%	Monitor demand closely
High	> 10%	Increase generation capacity

Project Structure

```
final_research/
│
├── backend/
│   ├── app.py                          # FastAPI application (all endpoints)
│   ├── requirements.txt                # Python dependencies
│   ├── .env                            # Environment configuration
│   ├── .env.example                    # Environment variable template
│   ├── saved_models/
│   │   ├── forecast_model.pkl          # Trained Random Forest model
│   │   ├── feat_scaler.pkl             # Feature StandardScaler
│   │   ├── target_scaler.pkl           # Target StandardScaler
│   │   └── model_metadata.json         # Model performance metrics & config
│   └── outputs/
│       └── scenario_results.json       # Pre-computed scenario analysis results
│
├── frontend/
│   ├── src/
│   │   ├── App.jsx                     # Root React component
│   │   ├── main.jsx                    # Vite application entry point
│   │   ├── index.css                   # Tailwind CSS styles
│   │   └── pages/
│   │       └── ProfessionalDashboard.jsx  # Main dashboard (4 tabs)
│   ├── public/
```

```
│   ├── package.json
│   ├── vite.config.js                  # Vite config with API proxy to backend
│   ├── tailwind.config.js
│   └── index.html
│
├── notebooks/
│   ├── 1_simple_model_training.ipynb   # Feature engineering + RF training
│   ├── 2_simple_scenario_analysis.ipynb # 4-scenario computation & export
│   └── load_forecasting_dataset_corrected.csv  # Raw training dataset
│
└── README.md
```

Setup & Installation

Prerequisites

- Python 3.9 or higher
- Node.js 18 or higher
- npm 9 or higher

---

1. Clone the Repository

```
git clone <repository-url>
cd final_research
```

2. Install Backend Dependencies

```
cd backend
pip install -r requirements.txt
```

3. Train the ML Model

Open and run all cells in the training notebook:

```
cd ../notebooks
jupyter notebook 1_simple_model_training.ipynb
```

This will: - Load and clean the raw dataset - Perform feature engineering (20 features) - Train the Random Forest model with Optuna hyperparameter optimisation - Save `forecast_model.pkl`, `feat_scaler.pkl`, `target_scaler.pkl` to `backend/saved_models/` - Save `model_metadata.json` with performance metrics

4. Run Scenario Pre-computation

```
jupyter notebook 2_simple_scenario_analysis.ipynb
```

This will: - Load the trained model - Compute 4 scenarios (Heat Wave, Cold Spell, Price Shock, Economic Boom) - Save results to `backend/outputs/scenario_results.json`

5. Start the Backend API

```
cd ../backend
uvicorn app:app --reload --host 0.0.0.0 --port 8002
```

6. Install and Start the Frontend

```
cd ../frontend
npm install
npm run dev
```

7. Access the System

Service	URL
React Dashboard	http://localhost:5175
API Base URL	http://127.0.0.1:8002
Swagger API Docs	http://127.0.0.1:8002/docs
Health Check	http://127.0.0.1:8002/api/health

API Reference

All endpoints are prefixed with `/api`. Backend runs on port **8002**.

Method	Endpoint	Description
`GET`	`/api/health`	System health check
`GET`	`/api/model-info`	Model architecture, performance metrics, and feature list
`POST`	`/api/predict`	Single demand prediction from 20 input features
`GET`	`/api/scenarios`	Retrieve pre-computed scenario analysis results
`POST`	`/api/scenario-test`	Run a custom scenario with arbitrary parameter modifications

Interactive API documentation (Swagger UI): **http://127.0.0.1:8002/docs**

Dashboard Features

The React frontend provides a 4-tab interactive dashboard:

Tab	Features
Overview	Model type, estimator count, training sample size, R^2 score, MAE, RMSE
Predict	Input form for all 20 features; real-time single demand prediction output
Scenarios	Side-by-side baseline vs. scenario comparison; percentage change; risk badge
Performance	Full model metrics; top feature importance summary

---

Notebooks

Notebook	Purpose
`1_simple_model_training.ipynb`	Feature engineering, outlier removal, RF training with Optuna, model export
`2_simple_scenario_analysis.ipynb`	Load model, compute 4 scenarios, export `scenario_results.json`

Environment Configuration

```
APP_NAME=Electricity Demand Forecasting - Scenario Analysis
APP_VERSION=1.0.0
DEBUG=False
MODEL_PATH=saved_models/forecast_model.pkl
LOG_LEVEL=INFO
```

License

This project is developed for academic research purposes at SLIIT. All rights reserved.